CachePIM: Reproducible Analytical Exploration of Cache-Aware CPU–Processing-in-Memory Execution

Aditya Singh Rana and Vinay Mor

School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, Tamil Nadu, India
aditya.singhrana2024@vitstudent.ac.in; vinay.mor2024@vitstudent.ac.in
Corresponding author: [To be confirmed by the authors before submission]

ABSTRACT

Processing-in-memory (PIM) places selected computation at or near memory to reduce data movement through the host memory hierarchy, but its modeled benefit depends on cache reuse, transfer overhead, and the assumed PIM throughput. This paper presents CachePIM, a reproducible, trace-driven analytical simulator that compares a CPU last-level-cache path with a bank-parallel PIM path. The simulator combines a fully associative least-recently-used cache, synthetic streaming, repeated-reuse, random, and mixed-phase traces, explicit host-to-PIM and PIM-to-host transfers, internal-memory service, compute throughput, launch and synchronization overheads, and bank-imbalance costs. Six execution policies are compared. The held-out evaluation comprises five master seeds, four workload patterns, five working-set sizes, and 600 policy/trace records. Under the default illustrative parameterization, mean modeled latency is 132.215 µs for CPU-cache and 22.190 µs for the PIM-selecting policies, a ratio of means of 5.96. These values are analytical estimates, not physical-hardware measurements. Reuse-policy tuning on separate development seeds selects a zero penalty; consequently, reuse-aware selection reduces to cost-based selection and does not improve it. Transfer and bank-count sweeps identify parameter-dependent crossover behavior, but the current sensitivity sweeps use one seed and are descriptive. The results therefore support a reproducible characterization of this model, not a claim of hardware speedup or a demonstrated advantage for the proposed heuristic.

INDEX TERMS— Analytical modeling, cache behavior, memory hierarchy, processing-in-memory, reproducible simulation, workload locality.

I. INTRODUCTION

Processing-in-memory (PIM) moves selected computation into or near the memory subsystem rather than requiring every operand to travel to a conventional host processor. The motivation is the cost of moving data in workloads whose arithmetic work is small relative to the volume of data accessed. PIM encompasses distinct designs, from computation that uses DRAM row-level operations to processing-near-memory engines with local cores or accelerators; the capabilities and execution costs therefore vary by architecture [1]. PIM is not universally faster. Offloading can incur host-to-device movement, device launch and synchronization overheads, limited local compute throughput, capacity restrictions, and coordination costs. The decision to execute on the host or in memory depends on both the workload and the target model [3].

CPU caches complicate this decision. Reuse in the last-level cache (LLC) can make host execution cheaper than a cold-memory access estimate suggests, whereas a large or streaming working set can generate repeated cache-line traffic. PIM can avoid some host-side accesses for the offloaded operation, but the inputs must be accessible at the PIM endpoint and results must be returned when needed. A policy that compares arithmetic throughput alone can omit these costs; conversely, a policy that ignores cache residency can overestimate CPU cost. Prior work has explicitly studied cache organization and cache-management policies for PIM systems [2], so the existence of a cache/PIM interaction is not claimed as new here.

This work studies that interaction through CachePIM, a software-only analytical simulator. It is designed to make model assumptions inspectable and allow repeated controlled experiments without access to physical PIM hardware. CachePIM uses synthetic address traces and a single fully associative LRU cache model, then estimates host and PIM execution costs using explicit component equations. Its policies include fixed CPU and PIM baselines, cost-based offloading, a reuse-aware candidate, and an oracle reference computed from actual simulator costs. The implementation and experiment results are available in the accompanying artifact; the public repository is https://github.com/AdityaRana101909/CachePIM-Analyser, although the corrected snapshot used for this manuscript had not yet been pushed to that repository when this draft was prepared [13].

The study asks three questions. RQ1: how does modeled CPU cache behavior change with working-set size and access pattern? RQ2: under which host-to-PIM transfer assumptions does the modeled PIM path cross the CPU-cache path? RQ3: does a reuse-aware penalty improve decisions over a cost-only rule under the implemented workload generator and cost equations? The third question is deliberately empirical: a heuristic is not treated as a contribution merely because it has an extra feature, and a negative result is reported when the selected penalty collapses to zero.

The contributions of this work are: (1) an inspectable, trace-driven cache/PIM cost model with explicit cache, transfer, memory-service, compute, launch, synchronization, and bank-imbalance terms; (2) an experiment harness that separates development tuning from held-out policy evaluation and aggregates traces at the master-seed level; (3) a controlled characterization of workload-size, transfer-bandwidth, bank-count, and reuse-penalty sensitivity; and (4) a documented negative result showing that the reuse-aware term does not improve the current model's cost-based selector. These are contributions of a compact analytical artifact and characterization study, not claims of cycle accuracy, calibrated hardware prediction, or a new general-purpose PIM compiler.

II. RELATED WORK

A. PIM systems and execution placement

PIM research addresses memory movement by placing computation at different levels of the memory hierarchy. The survey by Asifuzzaman et al. organizes PIM designs by memory technology, compute placement, application domain, and system integration challenges, including address mapping, workload suitability, runtime support, and coherence [1]. These architectural differences mean that the term PIM does not imply one universal latency model. Devic et al. study the decision of whether and how to offload general-purpose workloads to PIM engines in DDR memory systems, showing why parallelism, vectorization, dataset size, and offload cost matter [3]. CachePIM uses the same broad motivation—that execution location is a cost-sensitive decision—but implements only a transparent educational analytical abstraction.

B. Cache management in PIM systems

Kim et al. directly investigate cache-memory organization and management strategies for PIM-based computing systems, and compare alternative cache policies in modeled architectures [2]. This is the closest thematic antecedent to CachePIM's focus on the cache/PIM relationship. CachePIM therefore does not claim to be the first study of cache management in a PIM setting. Its focus is narrower and different: it treats host-cache trace behavior and the explicit transfer/compute terms of a separate PIM path as inputs to a reproducible offloading comparison, and it reports a held-out evaluation of an optional reuse-preservation penalty. The current evaluation does not show that the penalty improves the cost-only policy.

C. PIM compilers and execution simulators

SIMDRAM demonstrates a complete framework for lowering high-level operations into bit-serial operations supported by DRAM command sequences [4]. CHOPPER provides a compiler infrastructure for programmable bit-serial SIMD processing using DRAM [5]. PIMsynth integrates logic synthesis, PIM-specific transformation, scheduling, and code generation for bit-serial targets [9]. PIMCOMP addresses the end-to-end mapping of deep neural network models to configurable PIM accelerators [8]. These tools focus on compilation, operation synthesis, and target-specific execution support, which are beyond CachePIM's scope.

For performance evaluation, PIMSIM-NN combines an instruction-set abstraction, compiler, and configurable cycle-accurate simulator for neural-network PIM accelerators [6]. Siddique et al. describe architectural modeling and benchmarking for digital DRAM PIM [7]. These efforts provide richer architecture-oriented models than the current CachePIM abstraction. The recent VIPER preprint specifically proposes lightweight architecture-aware analytical performance modeling for PIM design-space

exploration and reports validation against a commercial system and cycle-accurate configurations [10]. Meta-PIM jointly explores data layouts, tiles, execution modes, and CPU/PIM placement [11], while DCC jointly considers data partitioning and compute partitioning for machine-learning kernels [12]. Compared with these systems, CachePIM deliberately omits instruction-level accuracy and hardware validation, and offers no claim to model fidelity comparable to them. Its purpose is to make cache reuse, transfer cost, and policy decisions visible in a small reproducible simulation.

D. Research scope and distinction

The literature establishes three boundaries for this paper. First, cache policy in PIM has been studied [2]. Second, the CPU/PIM decision and offload overheads have been investigated in architecture-aware frameworks [3], [10], [11]. Third, realistic PIM evaluation can require cycle-accurate simulation, calibrated models, or hardware measurements [6], [7], [10]. CachePIM does not attempt to supersede those methods. The study instead records what follows from one explicit set of analytical equations and synthetic access traces. Its useful output is a reproducible model-specific crossover map, including the conditions under which the proposed reuse-aware term fails to help. A broader novelty claim requires further comparison against the complete set of related systems and independent validation.

III. CACHEPIM ARCHITECTURE AND SYSTEM MODEL

CachePIM is organized as six loosely coupled components: (i) a seeded workload generator; (ii) a trace-driven LLC model; (iii) a CPU analytical cost model; (iv) a separate PIM analytical cost model; (v) a policy layer; and (vi) an evaluator and experiment runner. The dashboard uses the same model interfaces as the command-line experiment runner and is not a separate source of performance logic. Fig. 1 summarizes the execution flow.

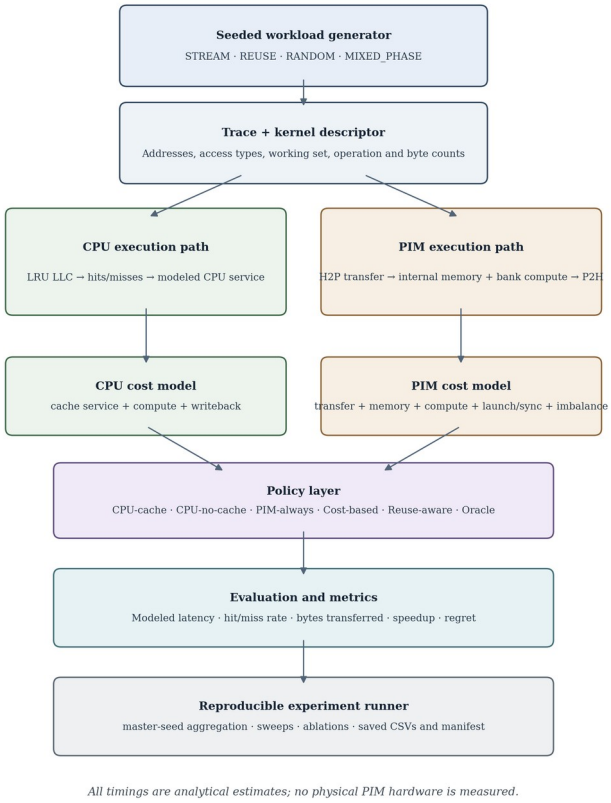


Fig. 1. CachePIM architecture. The trace generator feeds a modeled CPU-cache path and a separate PIM path. Policy selection and evaluation produce reproducible model-derived metrics; the framework does not measure physical hardware.

A. Workloads and traces

Each trace contains ordered memory references with a sequence number, byte address, read/write operation, task identifier, and phase identifier. The generator uses a fixed NumPy random-number generator seed. Four workload patterns are evaluated: STREAM accesses a contiguous array over two passes; REUSE targets a hot set with a configurable hot/cold ratio; RANDOM samples address locations over a defined working set; and MIXED_PHASE joins phases with different access characteristics. The operation descriptor records a kernel type, working-set size, element count, operation count, trace length, bytes read and written, PIM eligibility, and explicit PIM input/output footprints.

The implemented kernel metadata covers vector operations, reductions, and filters. For a vector operation with 64-bit elements, the descriptor models two input vectors and one output vector. Reduction models one input vector and one scalar output. Filter uses a conservative worst-case output volume equal to the input footprint. These are workload descriptors used by the timing model; the simulator does not run vector arithmetic on a PIM device. Trace semantics and transfer footprints are related but distinct: trace accesses drive cache statistics, while the operation descriptor determines the amount of data moved to and from PIM.

STREAM traces contain two contiguous passes and their length scales with working-set size. The other generated patterns use a configured access count (2,048 in the primary non-stream trace cases). Therefore, absolute latencies across workload classes are not a work-normalized comparison. Within each trace, policies see the same workload descriptor and access trace; comparisons against the corresponding CPU-cache baseline are meaningful for that trace under the model. The fixed synthetic workload generator is not a substitute for application traces.

B. Last-level cache model

The cache component models a single, fully associative LLC with capacity C bytes, line size L bytes, and exact least-recently-used (LRU) replacement. The cache-line identifier for byte address a is computed as floor(a/L), and the number of resident lines is floor(C/L). On a hit, the line is made most-recently used; on a miss, the line is inserted and the least-recently-used line is evicted if the cache is full. The implementation records accesses, hits, misses, evictions, dirty evictions, hit rate, miss rate, and data fetched from memory. Cache behavior is replayed from the trace; it is not hardcoded into workload metadata.

$$H = N_{hit} / N_{access}; \quad M = N_{miss} / N_{access}; \quad H + M = 1 \tag{1}$$

Here N_hit and N_miss denote the number of hit and miss events and N_access is the trace-access count; the rates are defined as zero for an empty trace. The identity in (1) is a test invariant for nonempty traces. In the initial study, the LLC is fully associative and uses LRU because the purpose is to isolate capacity and temporal/spatial locality effects without introducing indexing conflicts. Set associativity, multiple levels, prefetching, non-blocking misses, out-of-order execution, and cache-coherence protocol behavior are not modeled.

C. CPU cost model

The CPU model assigns a complete service time to each hit and miss. A miss-service time replaces the hit-service time for that access rather than being added to it; this avoids double-counting lookup latency. Computation is represented by a fixed time per operation. Dirty-line writeback is added when relevant according to the simulator's documented writeback accounting. The current primary workload descriptors are read-dominant and the study does not attempt to model a detailed coherence protocol for CPU-produced data consumed by PIM.

$$T_{CPU} = N_{hit}\, t_{hit} + N_{miss}\, t_{miss} + N_{op}\, t_{compute} + T_{writeback} \tag{2}$$

In (2), t_hit and t_miss are model parameters measured in nanoseconds, N_op is the operation count, and t_compute is the modeled CPU computation time per operation. The direct-memory CPU-no-cache policy provides a contrasting baseline in which each modeled access incurs memory service instead of benefiting from cache residency. This is a comparison policy, not a claim that a physical CPU has no cache.

D. PIM cost model

The PIM model separately accounts for external movement, internal-memory service, arithmetic, launch and synchronization overheads, and bank imbalance. When the input is not resident at the PIM endpoint, input movement is modeled as the byte count divided by effective host-to-PIM bandwidth plus a fixed transfer overhead. Output movement is defined analogously. The initial experiments assume that all kernel inputs begin outside PIM and charge the associated transfer cost. The model includes a flag for already-resident input, but no full hardware cache-coherence mechanism is represented; experiments must state the data-location assumption.

$$T_{H2P} = B_{in} / BW_{H2P} + F_{H2P} \tag{3}$$

$$T_{P2H} = B_{out} / BW_{P2H} + F_{P2H} \tag{4}$$

The fixed transfer overhead F is added only when its corresponding byte count is positive. Internal memory service is calculated from total kernel

input and output bytes and effective internal bandwidth. Computation is distributed across B PIM banks according to a deterministic workload-dependent rule. Let W_b be the operation count allocated to bank b and W_bar the average bank load. The imbalance factor is I = max_b(W_b)/W_bar for nonzero work, otherwise one. The base compute time is multiplied by bank count and per-bank throughput; the modeled imbalance penalty adds the additional time implied by I. The aggregate T_overhead equals T_launch + T_sync + T_imbalance. The implementation serially adds memory-service and compute terms, rather than overlapping them.

$$T_{mem} = (B_{in} + B_{out}) / BW_{internal} \quad (5)$$

$$T_{compute} = N_{op} / (B \cdot R_{bank}) \quad (6)$$

$$T_{PIM} = T_{H2P} + T_{mem} + T_{compute} + T_{P2H} + T_{overhead} \quad (7)$$

The equation is an analytical cost model and is not cycle-accurate. It does not represent DRAM command timing, row-buffer conflicts, queueing, detailed bank-group constraints, channel contention, device programming time, or overlapped execution. The term 'bank' in this simulator describes a configurable parallel service resource; the abstraction is not tied to one real PIM architecture. All default time and bandwidth values are illustrative assumptions unless an independent source and calibration procedure are reported.

E. Policy selection

Six policies are compared under the same trace. CPU-cache always selects the modeled cached CPU path; CPU-no-cache selects the direct-memory baseline; PIM-always selects PIM for supported operations and falls back to CPU when unsupported; Cost-based selects the smaller estimated cost; Reuse-aware adds a reuse-preservation penalty to estimated PIM cost; and Oracle selects the lower actual modeled cost after both paths are evaluated. Oracle is an offline reference and is not a deployable selection policy.

Cost-based and reuse-aware decisions use profile-level estimates from workload metadata rather than the exact trace-replay cost used to score the selected path. For the reuse-aware policy, estimated hit fraction gates the penalty: the penalty is not applied if the estimated hit fraction is below the configured threshold. Otherwise, the implementation forms an effective reusable-byte estimate from descriptor reuse, cache capacity, and working-set size. The corresponding time penalty is the normalized reusable fraction multiplied by the estimated CPU time benefit from hits.

$$T_{PIM,adj} = T_{PIM,est} + \lambda R_{cost} \quad (8)$$

The weight λ is dimensionless; R_cost is expressed in nanoseconds so that the adjusted estimate remains dimensionally consistent. The policy chooses PIM only when T_PIM,adj is lower than estimated CPU cost; ties select CPU. The tuning objective minimizes normalized regret on development master seeds, with modeled time and then smaller λ as tie-breaks. The selected value is frozen for held-out testing. It is important that λ = 0 makes (8) equal the cost-only policy; this is the outcome selected by the implemented tuning protocol, not a manually imposed paper assumption.

F. Evaluation metrics

Modeled execution time is the estimated cost of the path selected by the policy. Speedup is the CPU-cache modeled time divided by the selected policy time for the same trace. Regret compares selected-path actual model cost with the lower of actual CPU and PIM model costs. Bad-offload rate counts eligible cases in which the policy selects PIM although the model's actual CPU cost is lower. Cache hit rate, miss rate, transfer bytes, bank imbalance, and policy overhead are also recorded where applicable.

$$Speedup = T_{CPU\text{-}cache} / T_{selected} \quad (9)$$

$$Regret = T_{selected,actual} - \min(T_{CPU,actual}, T_{PIM,actual}) \quad (10)$$

These metrics describe the model. In particular, a speedup calculated from modeled nanoseconds is not a measurement of elapsed time on a CPU or PIM device. The oracle regret reference is useful for evaluating decision quality under the same cost equations; it does not show that the underlying equations match physical hardware.

IV. EXPERIMENTAL METHODOLOGY

A. Software stack and artifact

CachePIM is implemented in Python 3.10+ and uses NumPy for seeded workload generation, pandas for result tables and CSV export, Matplotlib for static figures, SciPy for statistical summaries, PyYAML for configuration files, Streamlit for the interactive dashboard, and pytest for tests. The dashboard and batch runner call the same simulation modules. Each result is linked to a configuration hash and model version. The corrected artifact is labelled 1.1.0-lambda-0. Results are written to distinct development, held-out, and sensitivity locations, and the manifest records seeds, configuration snapshots, source hashes, test status, and result classification.

B. Default parameterization

Table I summarizes the default model configuration. Values are included to make the experiment reproducible, not to identify a real CPU/PIM platform. The CPU time parameters, PIM bandwidth/throughput, and transfer terms are illustrative values. They have not been calibrated against physical hardware or fitted to a specific PIM simulator. The default LLC is 32 KiB with 64-byte lines. The PIM model has 16 banks, internal bandwidth of 128 bytes/ns, compute throughput of one operation/ns/bank, 50 ns launch overhead and 20 ns synchronization overhead. Host-to-PIM and PIM-to-host effective bandwidths are 16 bytes/ns, with 100 ns fixed overhead in each direction.

TABLE I. DEFAULT ANALYTICAL MODEL PARAMETERS (ASSUMED, NOT HARDWARE-CALIBRATED)

Component	Parameter	Default
LLC	Capacity; line; replacement	32 KiB; 64 B; LRU
CPU	Hit service; miss service	2 ns; 80 ns
CPU	Compute time per operation	0.4 ns/op
PIM	Banks; internal bandwidth	16; 128 B/ns
PIM	Compute throughput per bank	1 op/ns/bank
PIM	Launch; synchronization	50 ns; 20 ns
Transfer	Host to PIM bandwidth; fixed	16 B/ns; 100 ns
Transfer	PIM to host bandwidth; fixed	16 B/ns; 100 ns
Policy	Reuse threshold; selected λ	0.5; 0.0

The main held-out sweep varies working-set size across 16, 32, 64, 128, and 256 KiB, with four workload patterns. The LLC remains 32 KiB unless otherwise stated. These values create controlled relationships between capacity and working set, but they do not cover all possible cache geometries or PIM designs.

C. Development and held-out design

Master seeds 42–46 form the development pool; each contributes three traces per development case to select the reuse penalty. Master seeds 47–51 form the held-out pool; each contributes five trace indices mapped to five working-set sizes for each of the four workload patterns. The held-out experiment therefore includes 5 seeds × 4 patterns × 5 working-set cases = 100 trace cases. Six policies evaluated on every case produce 600 policy/trace rows. Trace outcomes are first averaged within master seed, and across-seed means and sample standard deviations are then calculated over five master-seed means. This aggregation avoids presenting individual traces from the same master-seed realization as independent master seeds.

Separate single-seed sweeps vary cache capacity and transfer bandwidth, transfer bandwidth alone, the number of PIM banks, bank-skew factor, and reuse penalty. The crossover and bandwidth sweeps reuse the same deterministic trace across parameter points to isolate parameter effects. These sweeps use development seed 42 and are descriptive, rather than inferential evidence about seed-to-seed variability. The final policy performance table below uses the five held-out seeds only.

D. Statistical treatment

The primary summary is mean ± sample standard deviation across the five held-out master-seed means. The sample standard deviation is descriptive. A Student-t interval is not interpreted as generalization beyond the synthetic workload generator and selected analytical assumptions. Paired comparisons use the same workload cases for each policy. For reuse-aware versus cost-based selection, all differences are exactly zero under the frozen λ = 0 setting; the corresponding t-statistic is degenerate and the result is reported as equivalence induced by the chosen parameter, not as evidence that two independently functioning algorithms are generally equivalent. Sensitivity plots involving one seed have no confidence interval and must not be treated as statistically representative.

V. RESULTS

A. Held-out baseline comparison

Table II reports the overall mean modeled latency, after averaging traces within each master seed. The CPU-cache baseline is 132.215 ± 0.159 µs. The direct-memory CPU baseline is 633.938 ± 0.000 µs. PIM-always, Cost-based, Reuse-aware, and Oracle all average 22.190 ± 0.000 µs. Thus, the ratio of mean CPU-cache time to mean PIM-selected time is 5.96 under this configuration and workload mix. All four policies select the same PIM path for all 100 held-out cases. The zero modeled standard deviation on the PIM-

selected policies follows from deterministic PIM cost equations and identical path selection; it does not indicate hardware stability.

TABLE II. HELD-OUT OVERALL POLICY COMPARISON

Policy	Mean latency (µs)	SD (µs)	Interpretation
CPU-cache	132.215	0.159	LLC trace replay
CPU-no-cache	633.938	0.000	Direct-memory baseline
PIM-always	22.190	0.000	Always offload eligible
Cost-based	22.190	0.000	Select estimated minimum
Reuse-aware	22.190	0.000	λ selected as zero
Oracle	22.190	0.000	Offline model reference

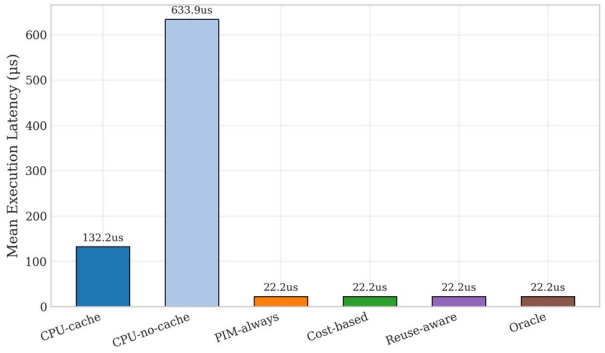


Fig. 2. Mean modeled latency across the held-out policy/trace cases. The figure compares analytical estimates under the default assumed parameters, not physical measurements.

The direct-memory baseline is slower than CPU-cache in this model because it forfeits modeled cache hits. This large difference is not a reliable estimate of the gain from adding a hardware cache: CPU-no-cache is deliberately a comparison baseline, not a calibrated CPU configuration. Similarly, the gap between CPU-cache and PIM reflects the assumed 16 B/ns host-PIM bandwidth, fixed transfer overheads, internal bandwidth, compute throughput, and the relatively high modeled CPU miss service time.

B. Cache locality and working-set size

Across the four patterns and held-out master seeds, the mean CPU-cache hit rate falls as the working set grows relative to the fixed 32 KiB LLC. Table III reports the average across workload patterns at each working-set size. The rate decreases from 89.40% at 16 KiB to 35.26% at 256 KiB. The trend is expected from the capacity relationship in this single-level cache model, although the detailed values depend on each generated address trace and access count.

TABLE III. MEAN CPU-CACHE HIT RATE BY WORKING-SET SIZE

Working set (KiB)	Mean hit rate	SD
16	89.40%	0.11 pp
32	81.66%	0.12 pp
64	64.54%	0.26 pp
128	46.42%	0.26 pp
256	35.26%	0.52 pp

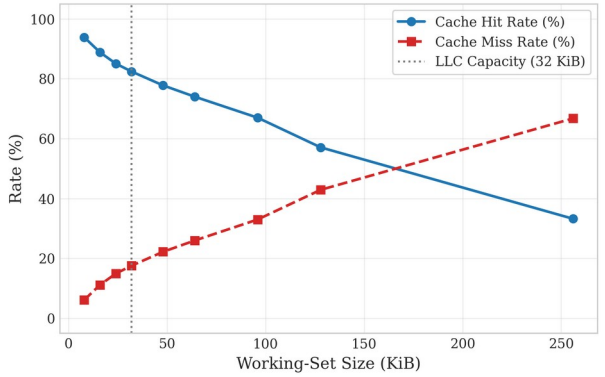


Fig. 3. Hit and miss rate versus working-set size from the saved cache sensitivity experiment. This sensitivity figure is descriptive and uses one development seed.

The pattern-level averages provide additional context. REUSE produces a mean hit rate of 67.57% ± 0.39 percentage points; RANDOM produces 47.64% ± 0.17 percentage points; MIXED_PHASE produces 48.61% ± 0.18 percentage points; and STREAM produces 90.00% in the current generator. The high STREAM hit rate should not be read as high temporal reuse alone: eight-byte sequential references share 64-byte lines, so spatial locality creates hits within a line, and the second pass can add further hits when capacity permits. Cache statistics are useful only when read with the generator semantics.

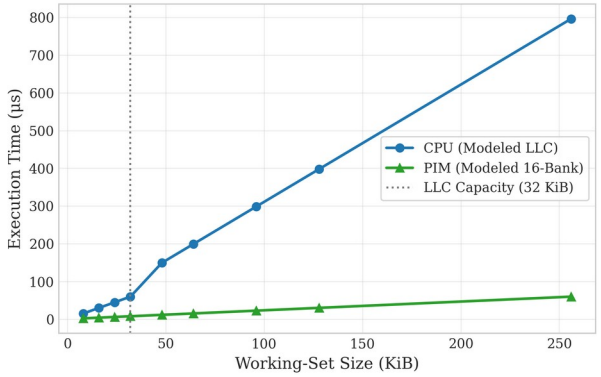


Fig. 4. Modeled CPU-cache and PIM time across working-set sizes under the default cost model. The graph characterizes this synthetic trace generator; it is not a hardware benchmark.

C. Workload-specific latency

Table IV aggregates each workload type across the five working-set cases and five held-out master seeds. The STREAM CPU-cache mean is 296.571 µs while its PIM mean is 23.284 µs, corresponding to a ratio of means of 12.74. The REUSE case is 56.723 µs on CPU-cache and 21.825 µs on PIM, ratio 2.60. RANDOM and MIXED_PHASE have corresponding ratios of 4.06 and 3.99. These within-pattern ratios are model outcomes, not comparisons at equal work across patterns: STREAM trace length scales with working-set size and number of passes, whereas other patterns use fixed trace lengths.

TABLE IV. MODELED LATENCY BY WORKLOAD PATTERN

Pattern	CPU-cache (µs)	PIM (µs)	Ratio
MIXED_PHASE	87.009 ± 0.295	21.825 ± 0	3.99×
RANDOM	88.556 ± 0.277	21.825 ± 0	4.06×
REUSE	56.723 ± 0.625	21.825 ± 0	2.60×
STREAM	296.571 ± 0	23.284 ± 0	12.74×

The fact that PIM cost is equal for several patterns in the table is a property of the model inputs: the configured kernel descriptors and byte footprints yield the same modeled cost for some trace types at the fixed defaults. STREAM differs because its descriptor's element and operation counts scale with working-set size. These outcomes should motivate examination of model structure and work normalization, not be interpreted as evidence that real hardware executes all random and reuse kernels in identical time.

D. Transfer-bandwidth sensitivity and crossover

The transfer sensitivity experiment holds the working set at 64 KiB, uses the default 32 KiB LLC and 16-bank PIM model, and varies host-to-PIM and PIM-to-host bandwidth together from 0.5 to 8 GB/s (decimal units). The same seed-42 trace is used for each bandwidth point. Table V reports the ratio of mean CPU-cache time to PIM-always time. At 0.5 GB/s, all three tested workload patterns are CPU-favorable in this model, with ratios below one. At 4 GB/s, STREAM and RANDOM become PIM-favorable, while REUSE remains slightly CPU-favorable at 0.91. At 8 GB/s, all three tested patterns favor PIM under the configured equations. The observed crossover differs by trace because the CPU path's cache behavior and the kernel descriptor's input/output footprint differ by workload type.

TABLE V. TRANSFER-BANDWIDTH SENSITIVITY (SPEEDUP = CPU-CACHE/PIM)

Bandwidth (GB/s)	STREAM	RANDOM	REUSE
0.5	0.503	0.243	0.117
1.0	0.998	0.483	0.234
2.0	1.968	0.956	0.463
4.0	3.830	1.877	0.908
8.0	7.264	3.616	1.751

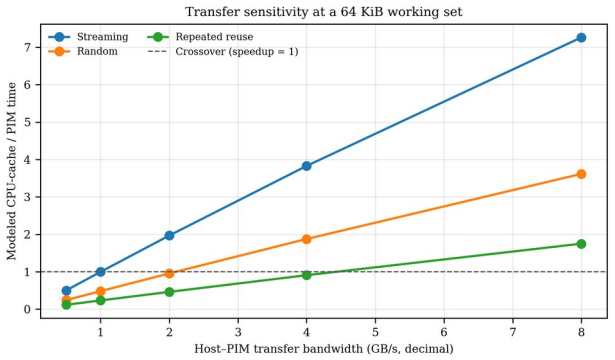


Fig. 5. Modeled CPU-cache/PIM ratio versus transfer bandwidth at a 64 KiB working set. The one-seed sweep is a parameter sensitivity result, not a multi-seed performance distribution.

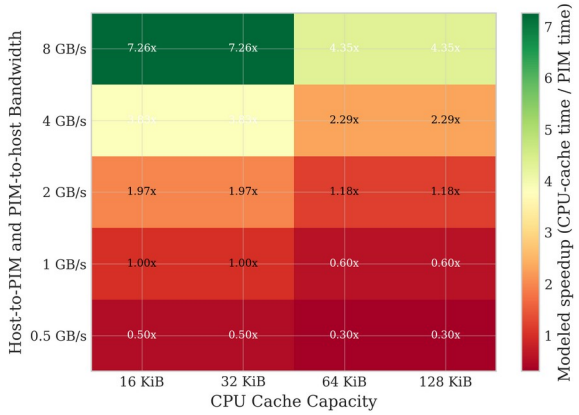


Fig. 6. Joint cache-capacity and transfer-bandwidth sensitivity for STREAM at a 64 KiB working set. The heatmap uses saved sweep outputs; values above one indicate lower modeled PIM time than CPU-cache time.

The ratio is not a universal crossover threshold. It depends on the assumed fixed transfer costs, bytes transferred, internal memory bandwidth, cache model, and kernel descriptor. The sweep shows why reporting PIM performance without movement costs can be misleading: at low transfer bandwidth the same PIM compute path may lose its modeled advantage. It does not establish that these particular bandwidth values correspond to a real interconnect or that the same crossover occurs on a physical device.

E. PIM bank-count and imbalance sensitivity

The bank-count sweep uses development seed 42 and averages ratios over the 16, 32, 64, and 128 KiB cases for each workload pattern. Increasing bank count from 4 to 64 improves the modeled CPU-cache/PIM ratio from 10.06× to 12.74× for STREAM, from 3.20× to 3.31× for REUSE, from 5.32× to 5.52× for RANDOM, and from 5.21× to 5.40× for MIXED_PHASE. The largest gains occur early, with diminishing gains at higher bank counts. This behavior follows from the PIM model: arithmetic service is divided by bank count while transfer and internal-memory terms do not scale proportionally with the bank count.

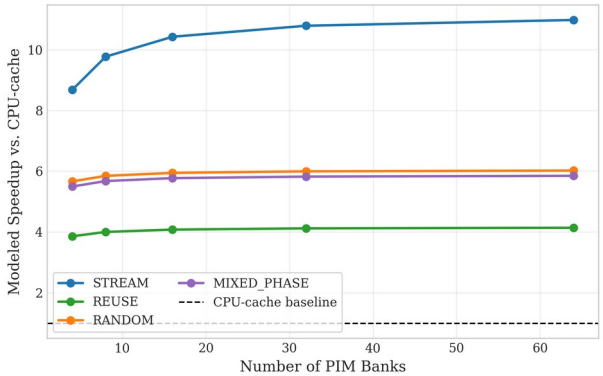


Fig. 7. Sensitivity of modeled CPU-cache/PIM ratio to bank count. Ratios are derived from a single development seed, across several working-set sizes, under the analytical model.

A separate skew sweep at 64 KiB and seed 42 increases the deterministic bank-work skew factor from 0 to 4. Mean PIM latency for STREAM rises from approximately 15.118 to 15.801 µs; RANDOM and REUSE rise from approximately 14.222 to 14.307 µs. The size of this effect is specific to the apportionment algorithm and the current operation counts. A real PIM device may have additional row, bank-group, channel, or controller constraints that this model does not represent.

F. Reuse-aware tuning and ablation

The reuse penalty is tuned on development master seeds 42–46 using three traces per seed. The selection objective minimizes mean normalized regret relative to the simulator's oracle, then uses selected modeled time and the lower penalty as tie-breaks. The selected value is λ = 0.0. Table VI shows the main development tuning outcomes. The selected policy therefore reduces exactly to Cost-based; it has no distinct performance behavior in the held-out evaluation. Positive penalties cause the reuse-aware selector to choose the more expensive host path in some cases under the default parameterization and increase normalized regret.

TABLE VI. REUSE-PENALTY TUNING ON DEVELOPMENT DATA

λ	Mean normalized regret	Mean regret (µs)	Mean selected time (µs)
0	0.000	0.000	8.579
0.25	2.116	21.806	30.385
0.5	2.116	21.806	30.385
1	2.994	26.655	35.235
1.5	2.994	26.655	35.235
2	3.694	30.347	38.926
3	3.694	30.347	38.926
4	3.694	30.347	38.926

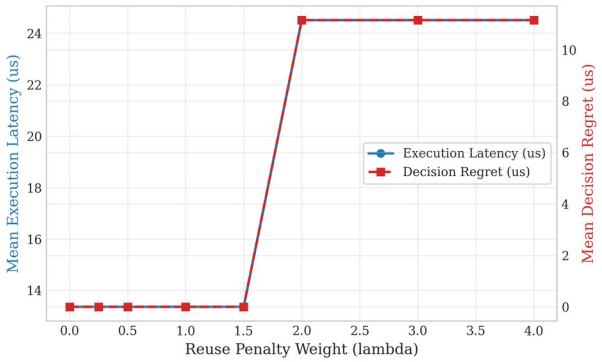


Fig. 8. Reuse-penalty ablation from saved development-seed results. Positive λ values raise regret in this configuration; λ=0 makes reuse-aware selection equivalent to cost-based selection.

On the held-out cases, the cost-based, reuse-aware, and oracle strategies all select PIM for every eligible trace and return identical overall modeled latency. The paired reuse-aware versus cost-based latency difference across five master seeds is exactly zero because λ = 0, and the test statistic is not informative beyond that equality. Thus RQ3 is answered negatively: the current reuse-aware rule does not demonstrate an improvement. The finding also exposes a limitation in the decision design. The cost-only estimate already predicts PIM to be cheaper for every held-out case, so an additive positive reuse penalty does not help unless the estimate is wrong in a way that the feature can correct.

G. Seed-level variability

The primary held-out policy summary aggregates within five master seeds. CPU-cache has 0.159 µs sample standard deviation across its master-seed means, while CPU-no-cache and the PIM-selected policies have zero modeled standard deviation under this dataset and deterministic equations. The held-out seed variability is summarized in Table II; a zero sample standard deviation for several policies follows from deterministic path costs rather than evidence of real hardware stability. The narrow error range should not be generalized: the PIM path's zero standard deviation is structural, because the model's input descriptors and cost parameters yield deterministic timing and policy selection. The five master seeds vary trace generation but do not represent independent real applications, devices, or operating environments.

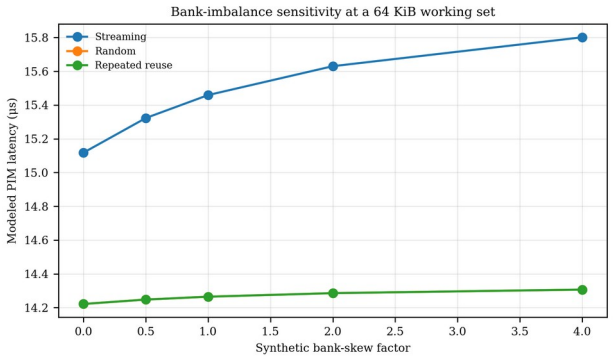


Fig. 9. Bank-skew sensitivity from the saved single-seed sweep. The deterministic skew factor is a synthetic mapping control, not a calibrated model of a physical DRAM bank-conflict mechanism.

VI. DISCUSSION

A. Interpretation of the 5.96 ratio

The primary result is a 5.96 ratio of mean modeled CPU-cache time to PIM-selected time under the default assumptions. This number summarizes the ratio produced by a chosen analytical model; it is not an empirical speedup. The result is driven by the values assigned to host cache miss service, operation cost, PIM compute throughput, internal memory bandwidth, transfer bandwidth and fixed overhead, and workload footprints. Because these values are illustrative rather than calibrated, the ratio should be read as an example of the model's behavior and a motivation for sensitivity analysis, not as a prediction for a named hardware platform.

A central finding from the transfer sweep is more robust within the model structure than the default ratio itself: adding a transfer term can reverse a PIM choice at low bandwidth, and different workload patterns cross the ratio-one boundary at different bandwidths. Nevertheless, since these sweeps use a single seed, the paper reports only deterministic parameter trends. Replicating each configuration across seeds would be necessary to quantify workload-generator variability in the crossover boundary.

B. Why reuse awareness did not help

The reuse-aware candidate was motivated by the idea that cache reuse can make CPU execution more attractive even when a cost-only estimate favors PIM. The implemented penalty instead increases estimated PIM cost in proportion to normalized reusable bytes and estimated CPU hit benefit. On development data, $\lambda = 0$ minimizes regret. This is a valid negative result for the implemented penalty and tested configuration: the added reuse term does not improve the selected policy. The study does not establish that all reuse-aware policies are ineffective. A future version could instead improve the underlying cache-cost estimate, model confidence/uncertainty explicitly, derive a phase-sensitive reuse benefit, or fit threshold choices against broader and more realistic traces. Any such extension must use training/development data only and be evaluated on held-out cases.

C. Threats to validity and limitations

The main threats to validity are: (1) parameter validity, because default timings and bandwidths are assumptions and are not calibrated to a physical CPU, interconnect, or PIM system; (2) model fidelity, because the LLC is a fully associative LRU cache and the PIM model serially adds memory and compute service; (3) workload representativeness, because the traces are generated synthetically and the STREAM trace length scales differently from the other patterns; (4) architectural generality, because no cache coherence, CPU-generated dirty data handoff, multi-level cache, row-buffer behavior, queueing, parallel outstanding misses, device programming, capacity partitioning, or concurrent system noise is represented; and (5) sweep uncertainty, because most sensitivity experiments use one master seed. The cost model also assumes simple bytes-per-bandwidth transfer time and fixed overheads, which may not capture protocol, burst, packet, synchronization, or setup effects of a real link.

The statistical uncertainty is narrow for some metrics because the simulator is deterministic and several policies choose identical paths. This is not evidence of low variance on real devices. The experimental unit is master seed, and the main held-out aggregate has five such units; this is a small synthetic evaluation. No external benchmark suite was used. No real CPU/PIM energy measurements were collected; the paper therefore does not report an energy-efficiency conclusion. It is also not a cycle-accurate simulator and should not be compared as equivalent to instruction-set-driven or hardware-validated PIM simulators [6], [7], [10].

D. Reproducibility and artifact status

The corrected local artifact includes source modules, configuration files, automated tests, raw CSV results, aggregated summaries, sensitivity figures in PNG/PDF, and a JSON manifest with model version and file hashes. The test suite passed 24 tests and the full experiment runner exited successfully for the reported artifact. The held-out evaluation uses master seeds 47–51, four workload patterns, five working-set sizes, and six policies (600 policy/trace records). The exact corrected artifact was labelled 1.1.0-lambda-0 and had not yet been pushed to the public GitHub repository at manuscript preparation time. Before publication, the authors should release the exact version used for the paper and replace the repository pointer with a permanent tagged release or archive DOI.

VII. CONCLUSION

This paper presented CachePIM, a small trace-driven analytical simulator for examining the interaction between host-cache behavior and CPU/PIM execution placement. The implementation models a fully associative LRU LLC, separate CPU and PIM cost equations, host-to-PIM and PIM-to-host movement, internal memory service, bank parallelism and imbalance, and six policy alternatives. Under the default assumed configuration, mean modeled latency is 132.215 µs for CPU-cache and 22.190 µs for the policies that choose PIM, a 5.96 ratio of means across the held-out synthetic workload mix. Transfer-bandwidth experiments show that the modeled crossover depends on data movement and access pattern. Increasing bank count produces diminishing modeled gains under the model's serial transfer/memory/compute accounting.

The reuse-aware policy does not improve on cost-based selection: development tuning chooses $\lambda = 0$, causing the two policies to coincide. This negative result is reported rather than obscured. The findings are strictly about the implemented analytical model and synthetic traces; they do not establish actual CPU/PIM speedup, real energy savings, or predictive accuracy for a device. The next steps are to validate model trends against a compatible external simulator or published measurements, repeat sensitivity sweeps across multiple master seeds, normalize workloads by useful work where appropriate, model additional cache/coherence behavior only when justified, and publish a permanent versioned artifact. CachePIM should be treated as a reproducible starting point for studying assumptions, not as a replacement for architecture-calibrated evaluation.

ACKNOWLEDGMENT

The authors report no external funding and have no additional acknowledgments to declare.

DATA AND CODE AVAILABILITY

The project repository is available at https://github.com/AdityaRana101909/CachePIM-Analyser [13]. The corrected `1.1.0-lambda-0` artifact and its experiment outputs accompany this manuscript, but the exact corrected snapshot had not yet been pushed to the public repository at the time of writing. Before submission, this section should point to a permanent release or archival DOI and include the exact commit identifier associated with all reported CSVs and figures.

REFERENCES

[1] K. Asifuzzaman, N. R. Miniskar, A. R. Young, F. Liu, and J. S. Vetter, “A survey on processing-in-memory techniques: Advances and challenges,” Memories - Materials, Devices, Circuits and Systems, vol. 4, Art. no. 100022, Jul. 2023, doi: 10.1016/j.memori.2022.100022.

[2] Y.-K. Kim, B. Moon, D.-S. Kim, and Y.-J. Jang, “Cache memory organization for processing in memory,” IEICE Electronics Express, vol. 16, no. 16, Art. no. 20190393, 2019, doi: 10.1587/ELEX.16.20190393.

[3] A. Devic, S. B. Rai, A. Sivasubramaniam, A. Akel, S. Eilert, and J. Eno, “To PIM or not for emerging general purpose processing in DDR memory systems,” in Proc. 49th Annu. Int. Symp. Computer Architecture (ISCA), 2022, pp. 231–244, doi: 10.1145/3470496.3527431.

[4] N. Hajinazar, G. F. Oliveira, S. Gregorio, J. Ferreira, N. M. Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, and O. Mutlu, “SIMDRAM: An end-to-end framework for bit-serial SIMD computing in DRAM,” in Proc. 26th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021, doi: 10.1145/3445814.3446749.

[5] X. Peng, Y. Wang, and M.-C. Yang, “CHOPPER: A compiler infrastructure for programmable bit-serial SIMD processing using memory in DRAM,” in Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA), 2023, pp. 1275–1288, doi: 10.1109/HPCA56546.2023.10071070.

[6] X. Wang, X. Sun, Y. Han, and X. Chen, “PIMSIM-NN: An ISA-based simulation framework for processing-in-memory accelerators,” in Proc. Design, Automation & Test in Europe Conf. & Exhibition (DATE), 2024, doi: 10.23919/DATE58400.2024.10546788.

[7] F. A. Siddique et al., “Architectural modeling and benchmarking for digital DRAM PIM,” in Proc. IEEE Int. Symp. Workload Characterization (IISWC), 2024, doi: 10.1109/IISWC63097.2024.00030.

[8] X. Sun, X. Wang, W. Li, Y. Han, and X. Chen, “PIMCOMP: An end-to-end DNN compiler for processing-in-memory accelerators,” IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, vol. 44, no. 5, pp. 1745–1759, 2025, doi: 10.1109/TCAD.2024.3496847.

[9] D. Guo, M. Gholamrezaei, M. Hofmann, A. Venkat, Z. Zhang, and K. Skadron, “PIMsynth: A unified compiler framework for bit-serial processing-in-memory architectures,” IEEE Computer Architecture Letters, vol. 24, no. 2, pp. 277–280, 2025, doi: 10.1109/LCA.2025.3600588.

[10] H. Geng, T. S. Pereira, X. Lu, X.-H. Sun, M. Niemier, and X. S. Hu, “VIPER: Architecture-aware performance modeling for processing-in-memory design-space exploration,” arXiv:2608.23404, Aug. 2026. [Online]. Available: https://arxiv.org/abs/2608.23404. (Preprint; not treated here as peer-reviewed publication.)

[11] J. Zhang, H. Wang, and E. Sadredini, “Meta-PIM: An end-to-end compiler for configurable processing-in-memory,” IEEE Computer Architecture Letters, early access, Jul. 2026, doi: 10.1109/LCA.2026.3709790.

[12] P. Yang, S. Durvasula, I. Fernandez, M. Sadrosadati, O. Mutlu, G. Pekhimenko, and C. Giannoula, “DCC: Data-centric compilation of machine learning kernels for processing-in-memory architectures,” in Proc. 53rd ACM/IEEE Annu. Int. Symp. Computer Architecture (ISCA), 2026, pp. 2582–2599, doi: 10.1109/ISCA66397.2026.00180.

[13] A. S. Rana and V. Mor, “CachePIM-Analyser: A reproducible analytical simulator for cache-aware CPU–PIM execution,” source repository, 2026. [Online]. Available: https://github.com/AdityaRana101909/CachePIM-Analyser. The corrected artifact version used in this manuscript is 1.1.0-lambda-0 and was not yet published to this URL at manuscript preparation time.